TECHNICAL DOCUMENT

Security Architecture

docs/SECURITY_ARCHITECTURE.md

JWT RS256 · Argon2id · Zero Trust · OWASP

PLATFORM	STACK	GENERATED
EduTech v1.0	Java 17 · Spring Boot 3.3.5	March 07, 2026

Security Architecture — EduTech AI Platform

Last updated: 2026-03-07
Classification: Internal — Engineering Reference

Table of Contents

1. Security Model Overview
2. JWT Token Architecture
3. API Gateway Security
4. Password Security
5. Bot Protection
6. OTP (One-Time Password)
7. Rate Limiting
8. Transport Security
9. Database Security
10. Security Headers and Request Tracing
11. Audit Trail
12. Threat Model Summary

1. Security Model Overview

The EduTech AI Platform follows a **zero-trust perimeter** design: every inbound request is verified at the `api-gateway` edge before it reaches any downstream service. No downstream service trusts client-supplied identity claims.

Core principles:

- **Asymmetric signing (RS256):** Access tokens are signed with an RSA private key held exclusively by `auth-svc`. All other services validate tokens using the corresponding public key — they can verify but never forge tokens.
- **Device-fingerprint binding:** Every token is cryptographically bound to the device that authenticated. A token stolen from one device cannot be replayed from another.
- **Short-lived access tokens:** 15-minute TTL limits the blast radius of token compromise without requiring revocation infrastructure.
- **Single-use refresh tokens:** Each token rotation consumes and replaces the previous refresh token. Replay of a consumed refresh token is detectable.
- **Role-based access control (RBAC):** Authorization is enforced via JWT claims. Roles follow a strict hierarchy:

```
SUPER_ADMIN → CENTER_ADMIN → TEACHER → PARENT → STUDENT
```

A user's role is set at registration time and embedded in the signed JWT. Clients cannot elevate their own role — the gateway enforces this by overwriting any client-supplied `X-User-Role` header with the value extracted from the validated token.

---

2. JWT Token Architecture

Access Token (15-minute TTL by default)

Access tokens use **RS256** (RSA Signature with SHA-256). The private key is stored as a PKCS#8 PEM file on the `auth-svc` host; the path is injected via `JWT_PRIVATE_KEY_PATH`. The key is loaded once at startup by `JwtTokenProvider` via `@PostConstruct`.

Token claims:

```
{
  "sub": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "email": "teacher@brightacademy.com",
  "role": "TEACHER",
  "centerId": "c0ffee00-dead-beef-cafe-000000000001",
  "deviceFP": "sha256:e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855",
  "jti": "550e8400-e29b-41d4-a716-446655440000",
  "iss": "https://edutech.com/auth",
  "iat": 1741338600,
```

```
  "exp": 1741339500
}
```

CLAIM	DESCRIPTION
`sub`	User UUID (auth-svc primary key)
`email`	User's email address (informational; not used for auth decisions)
`role`	RBAC role string, one of: `SUPER_ADMIN`, `CENTER_ADMIN`, `TEACHER`, `PARENT`, `STUDENT`
`centerId`	UUID of the user's associated center; absent for `SUPER_ADMIN`
`deviceFP`	SHA-256 hash of device fingerprint; bound at login time
`jti`	JWT ID — random UUID per token; enables precise single-token revocation
`iss`	Issuer URI, validated by all token validators
`iat`	Issued-at timestamp (Unix seconds)
`exp`	Expiry timestamp (Unix seconds); default TTL: `JWT_ACCESS_TOKEN_EXPIRY_SECONDS`

Token generation (`JwtTokenProvider`):

```
return Jwts.builder()
    .subject(user.getId().toString())
    .claim("email", user.getEmail())
    .claim("role", user.getRole().name())
    .claim("centerId", user.getCenterId() != null ? user.getCenterId().toString() : null)
    .claim("deviceFP", deviceFingerprintHash)
    .id(UUID.randomUUID().toString())
    .issuer(jwtProperties.issuer())
    .issuedAt(Date.from(now))
    .expiration(Date.from(expiry))
    .signWith(privateKey)   // RS256 via JJWT 0.12.x — algorithm inferred from key type
    .compact();
```

Refresh Token (7-day TTL by default)

Refresh tokens are **not JWTs** — they are opaque random identifiers stored and managed in Redis.

Lifecycle:

1. On successful login, a random `tokenId` (UUID) is generated.
2. The token is stored in Redis:
3. `rt:{tokenId}` → serialized token data (userId, deviceFP, expiry) — TTL: `JWT_REFRESH_TOKEN_EXPIRY_SECONDS`
4. `rt:user:{userId}` (Redis Set) → contains `tokenId`; used for logout-all

5. On refresh (`POST /api/v1/auth/refresh`):
6. `rt:{oldTokenId}` is **deleted atomically** (single-use enforcement)
7. Device fingerprint from the stored token is compared against the request's fingerprint — mismatch = HTTP 401
8. A new `(tokenId, accessToken)` pair is issued
9. `TokenRefreshedEvent` is published with both `oldTokenId` and `newTokenId`
10. On logout:
11. Single session: `rt:{tokenId}` deleted; entry removed from `rt:user:{userId}` set
12. All sessions: all tokenIds in `rt:user:{userId}` deleted, then the set itself is deleted

Redis key schema:

KEY PATTERN	TYPE	CONTENT	TTL
`rt:{tokenId}`	String	Serialized token data (userId, deviceFP, expiry)	`JWT_REFRESH_TOKEN_EXPIRY_SECONDS`
`rt:user:{userId}`	Set	All active tokenId values for this user	No TTL (managed by application logic)

JWT Validation

All services that need to validate JWTs (api-gateway, auth-svc, center-svc, parent-svc, assess-svc, psych-svc, ai-gateway-svc) load the RSA public key from `JWT_PUBLIC_KEY_PATH` at startup. The validation logic from `JwtTokenValidator`:

```
Claims claims = Jwts.parser()
    .verifyWith(publicKey)            // RSA public key — cannot forge, only verify
    .requireIssuer(jwtProperties.issuer())
    .build()
    .parseSignedClaims(token)
    .getPayload();
```

A `JwtException` or `IllegalArgumentException` from this call means the token is invalid, expired, or tampered. The caller receives `Optional.empty()` and the request is rejected with HTTP 401.

---

3. API Gateway Security

The `api-gateway` service (Spring Cloud Gateway, reactive WebFlux) is the single ingress point for all external traffic.

Filter Execution Order

ORDER	FILTER	ROLE
`-2`	`RequestIdFilter`	Generates `X-Request-ID` if absent
`-1`	`JwtAuthenticationFilter`	Validates JWT, injects identity headers

`JwtAuthenticationFilter` (GlobalFilter, order = -1)

Implemented as a `GlobalFilter` (not a route-level filter) so it applies unconditionally to every request before routing.

Public paths (bypass JWT validation):

```
/api/v1/auth/**
/actuator
/swagger-ui
/v3/api-docs
/webjars
```

For all other paths:

1. Extract `Authorization: Bearer <token>` header. If absent or malformed → HTTP 401, request terminates.
2. Validate token via `JwtTokenValidator.validate(token)` (JJWT RS256 verification + issuer check).
3. If invalid → HTTP 401, request terminates. The failure is logged at WARN level with path and error message.
4. If valid → mutate the downstream request to include:
5. `X-User-Id`: value of `claims.getSubject()` (user UUID)
6. `X-User-Role`: value of `claims.get("role")` (role string)
7. `X-User-Center-Id`: value of `claims.get("centerId")` (center UUID, if present)
8. Forward the mutated request to the downstream service.

Header injection (from the actual filter implementation):

```
ServerHttpRequest.Builder requestBuilder = exchange.getRequest().mutate()
    .header("X-User-Id", claims.getSubject());
String role = claims.get("role", String.class);
if (role != null) {
    requestBuilder.header("X-User-Role", role);
}
String centerId = claims.get("centerId", String.class);
if (centerId != null) {
    requestBuilder.header("X-User-Center-Id", centerId);
}
```

Downstream Trust Model

Downstream services trust `X-User-Id`, `X-User-Role`, and `X-User-Center-Id` headers implicitly — they do not re-validate the JWT. This is safe because:

- All external traffic passes through the gateway
- Services are not directly exposed to the public network
- The gateway strips any client-supplied `X-User-*` headers before forwarding (by mutating the request object)

Route Table

ROUTE ID	PATH PREDICATE	UPSTREAM SERVICE
`auth-svc`	`/api/v1/auth/**`	`AUTH_SVC_URI`
`parent-svc`	`/api/v1/parents/**`	`PARENT_SVC_URI`
`center-svc`	`/api/v1/centers/**`	`CENTER_SVC_URI`
`assess-svc`	`/api/v1/assessments/**`	`ASSESS_SVC_URI`
`psych-svc`	`/api/v1/psych/**`	`PSYCH_SVC_URI`
`ai-gateway-svc`	`/api/v1/ai/**`	`AI_GATEWAY_SVC_URI`

auth-svc Internal Filter

`auth-svc` runs its own `JwtAuthenticationFilter` (servlet-based `OncePerRequestFilter`) for endpoints that require authentication within auth-svc itself (e.g., change-password, logout). It extracts the Bearer token, validates it, and sets the Spring Security `SecurityContextHolder` principal.

Public endpoints in auth-svc's own `SecurityConfig`:

```
POST /api/v1/auth/register
POST /api/v1/auth/login
POST /api/v1/auth/refresh
POST /api/v1/otp/send
POST /api/v1/otp/verify
GET  /actuator/health
GET  /actuator/info
GET  /api-docs/**
GET  /swagger-ui/**
```

The filter also sets MDC context keys (`userId`, `role`) for structured log correlation within auth-svc's request scope.

---

4. Password Security

All user passwords are hashed using **Argon2id** — the OWASP-recommended algorithm for password hashing as of 2024.

Implementation

Spring Security's `Argon2PasswordEncoder` is used, wired through the `Argon2PasswordHasherAdapter` which implements the `PasswordHasher` domain port. This keeps the domain layer free of Spring Security dependencies.

```
// SecurityConfig.java — bean definition
@Bean
public Argon2PasswordEncoder argon2PasswordEncoder() {
    return new Argon2PasswordEncoder(
        argon2Properties.saltLength(),
        argon2Properties.hashLength(),
        argon2Properties.parallelism(),
        argon2Properties.memoryCost(),
        argon2Properties.iterations()
    );
}
```

Configuration Parameters

All Argon2 parameters are externalized — there are no in-code defaults. This is intentional: a misconfigured or missing environment variable causes a startup failure rather than silently using a weak configuration.

PARAMETER	ENVIRONMENT VARIABLE	OWASP 2024 RECOMMENDED
Memory cost (KB)	`ARGON2_MEMORY_COST`	65536 (64 MB)
Iterations	`ARGON2_ITERATIONS`	3
Parallelism	`ARGON2_PARALLELISM`	2
Salt length (bytes)	`ARGON2_SALT_LENGTH`	16
Hash length (bytes)	`ARGON2_HASH_LENGTH`	32

Config record (`Argon2Properties`):

```
@ConfigurationProperties(prefix = "argon2")
public record Argon2Properties(
    int memoryCost,
    int iterations,
    int parallelism,
    int saltLength,
    int hashLength
) {}
```

Security Properties

- A unique random salt is generated per password hash (handled by Spring's `Argon2PasswordEncoder`).
- Raw passwords are never logged, stored, or transmitted after the hashing step.
- Password verification uses `encoder.matches(rawPassword, storedHash)` — constant-time comparison is handled by the Bouncy Castle library underlying Spring Security's Argon2 implementation.

---

5. Bot Protection

hCaptcha Enterprise

Server-side hCaptcha verification is applied to registration and other high-risk actions to prevent automated account creation and credential stuffing at scale.

Verification flow:

1. Client solves the hCaptcha challenge and obtains a response token.
2. Client submits the token alongside the request payload.
3. `auth-svc` makes a server-to-server POST to the hCaptcha verify endpoint:
4. URL: `HCAPTCHA_VERIFY_URL` (e.g., `https://hcaptcha.com/siteverify`)
5. Payload: `response=<client-token>&secret=<HCAPTCHA_SECRET_KEY>`
6. If verification fails or the hCaptcha service is unavailable, the request is rejected.

Resilience: The hCaptcha HTTP client is protected by a Resilience4j circuit breaker (`captcha-client` instance). If the captcha service becomes unavailable (circuit open), the configured fallback behavior applies — by default this fails the request to prevent bot bypass during outages.

Configuration:

PROPERTY	ENVIRONMENT VARIABLE
Verify URL	`HCAPTCHA_VERIFY_URL`
Site key (public)	`HCAPTCHA_SITE_KEY`
Secret key (private)	`HCAPTCHA_SECRET_KEY`

The `HCAPTCHA_SECRET_KEY` is a sensitive credential and must never be exposed in client-side code, logs, or API responses.

---

6. OTP (One-Time Password)

OTPs are used for email verification, password reset, and two-factor login challenges.

Redis Storage Schema

KEY PATTERN	TYPE	VALUE	TTL
`otp:{email}:{purpose}`	String	The OTP value (numeric string)	`OTP_EXPIRY_SECONDS`
`otp:{email}:{purpose}:att`	String	Attempt count (integer)	`OTP_EXPIRY_SECONDS`

OTP Purposes

PURPOSE	USE CASE
`EMAIL_VERIFICATION`	Verify email address on new account registration
`PASSWORD_RESET`	Confirm identity before allowing password change
`LOGIN_2FA`	Second factor during login for high-privilege roles

Configuration

All OTP parameters are externalized via `OtpProperties`:

```
@ConfigurationProperties(prefix = "otp")
public record OtpProperties(
    int expirySeconds,    // OTP_EXPIRY_SECONDS
    int maxAttempts,      // OTP_MAX_ATTEMPTS
    int length            // OTP_LENGTH
) {}
```

PARAMETER	ENVIRONMENT VARIABLE	TYPICAL VALUE
Expiry	`OTP_EXPIRY_SECONDS`	300 (5 minutes)
Max failed attempts	`OTP_MAX_ATTEMPTS`	3
OTP digit length	`OTP_LENGTH`	6

Security Properties

- The OTP value is **never included** in any Kafka event (including `OtpRequestedEvent`). Only the notification pipeline knows the actual OTP value, delivered via a separate in-process call before the event is published.
- After `maxAttempts` failed verifications, the OTP key is deleted from Redis, forcing the user to request a new OTP.
- OTP verification is rate-limited by the Resilience4j `login` rate limiter at the application layer in addition to the gateway-level token bucket.
- OTPs are generated using a cryptographically secure random number generator (`SecureRandom`).

---

7. Rate Limiting

Two independent rate-limiting layers operate in the platform.

Layer 1: API Gateway — Token Bucket (per-IP or per-user)

Spring Cloud Gateway's built-in `RequestRateLimiter` filter is applied as a **default filter** to all routes. It uses Redis to implement a token bucket algorithm.

Configuration (from `api-gateway/application.yml`):

```
spring:
  cloud:
    gateway:
      default-filters:
        - name: RequestRateLimiter
          args:
            redis-rate-limiter.replenish-rate: ${GATEWAY_RATE_LIMIT_REPLENISH_RATE}
            redis-rate-limiter.burst-capacity: ${GATEWAY_RATE_LIMIT_BURST_CAPACITY}
            redis-rate-limiter.requested-tokens: ${GATEWAY_RATE_LIMIT_REQUESTED_TOKENS}
```

ENVIRONMENT VARIABLE	PURPOSE
`GATEWAY_RATE_LIMIT_REPLENISH_RATE`	Tokens added per second (sustained request rate)
`GATEWAY_RATE_LIMIT_BURST_CAPACITY`	Maximum token bucket depth (burst allowance)
`GATEWAY_RATE_LIMIT_REQUESTED_TOKENS`	Tokens consumed per request (default: 1)

Requests that exceed the limit receive HTTP 429 with a `Retry-After` header.

Layer 2: AI Gateway — Per-Requester Sliding Window

`ai-gateway-svc` applies a finer-grained per-requester rate limit using `RedisRateLimitAdapter`, keyed by requester identity and model type.

Redis key: `ratelimit:{requesterId}:{modelType}`

Algorithm: Atomic increment with a 60-second TTL window. On the first request in a window, `INCR` is followed by `EXPIRE 60`. Subsequent increments within the window are allowed up to the limit.

```
// RedisRateLimitAdapter.java
String key = "ratelimit:" + requesterId + ":" + modelType.name();
return redisTemplate.opsForValue().increment(key)
    .flatMap(count -> {
        if (count == 1L) {
            return redisTemplate.expire(key, Duration.ofSeconds(60))
                    .thenReturn(true);
        }
        return Mono.just(count <= MAX_REQUESTS_PER_MINUTE); // 60 req/min
```

```
    })
    .onErrorReturn(true); // Fail open on Redis outage — allows the request
```

Fail-open behavior: If Redis is unavailable, the rate limiter returns `true` (allows the request). This is a deliberate availability-over-security tradeoff for AI requests. An outage alert should be triggered on Redis connectivity failure.

PARAMETER	VALUE
Max requests per minute	60
Window duration	60 seconds
Key scope	Per requester + per model type

8. Transport Security

LAYER	MECHANISM	STATUS
External client → api-gateway	HTTPS (TLS 1.2+)	Production enforced
Inter-service (internal)	HTTPS	Production enforced
TLS termination	At api-gateway	Active
Service mesh mTLS	Istio (mutual TLS)	Planned for production
Redis connection	SSL/TLS configurable	`REDIS_SSL_ENABLED` env var

Redis SSL configuration (identical across all services):

```
spring:
  data:
    redis:
      ssl:
        enabled: ${REDIS_SSL_ENABLED}
```

In local development, `REDIS_SSL_ENABLED=false`. In production, this must be `true` to protect credentials and token data in transit.

9. Database Security

Per-Service Credentials

Each service has its own isolated database credentials. No service has access to another service's database schema. Credentials are injected via environment variables:

SERVICE	DB USERNAME VARIABLE	DB PASSWORD VARIABLE	SCHEMA
auth-svc	`AUTH_SVC_DB_USERNAME`	`AUTH_SVC_DB_PASSWORD`	`auth_schema`
center-svc	`CENTER_SVC_DB_USERNAME`	`CENTER_SVC_DB_PASSWORD`	`center_schema`
parent-svc	`PARENT_SVC_DB_USERNAME`	`PARENT_SVC_DB_PASSWORD`	`parent_schema`
assess-svc	`ASSESS_SVC_DB_USERNAME`	`ASSESS_SVC_DB_PASSWORD`	`assess_schema`
psych-svc	`PSYCH_SVC_DB_USERNAME`	`PSYCH_SVC_DB_PASSWORD`	`psych_schema`

Soft Delete

No domain entity is physically deleted from the database. All deletions are soft deletes using a `deleted_at` column (`IS NULL` in all active record queries). This preserves data for:

- Audit trail reconstruction
- Regulatory compliance (data retention requirements)
- Forensic investigation

Optimistic Locking

JPA `@Version` fields are used on all mutable aggregate roots to prevent lost updates in concurrent scenarios. A stale write attempt raises `OptimisticLockException`, which is handled at the application layer.

SQL Injection Prevention

All database interactions use JPA with JPQL named queries or Spring Data repository methods. There is no string concatenation for SQL construction anywhere in the codebase. Raw JDBC or native SQL queries are not used in application code.

Schema Management

Database schema migrations are managed by **Flyway**, with per-service migration paths:

```
flyway:
  schemas: <service>_schema
  locations: classpath:db/migration/<service>
  baseline-on-migrate: false
```

`baseline-on-migrate: false` ensures that Flyway will fail fast on a non-empty, non-baselined database rather than silently skipping migration history validation.

---

10. Security Headers and Request Tracing

`RequestIdFilter` (api-gateway, order = -2)

Runs before the JWT filter. If the inbound request does not carry an `X-Request-ID` header, a random UUID is generated and injected. This header propagates through the entire request chain:

```
// RequestIdFilter.java
if (requestId == null || requestId.isBlank()) {
    ServerHttpRequest mutated = exchange.getRequest().mutate()
            .header("X-Request-ID", UUID.randomUUID().toString())
            .build();
    return chain.filter(exchange.mutate().request(mutated).build());
}
```

This enables end-to-end distributed tracing correlation without relying on the client to supply a trustworthy ID.

Gateway-Injected Headers (downstream only)

The following headers are set exclusively by the api-gateway after JWT validation. Downstream services must not accept these headers from clients directly.

HEADER	SOURCE	CONTENT
`X-Request-ID`	`RequestIdFilter`	Random UUID or client-supplied ID
`X-User-Id`	`JwtAuthenticationFilter`	`sub` claim from validated JWT
`X-User-Role`	`JwtAuthenticationFilter`	`role` claim from validated JWT
`X-User-Center-Id`	`JwtAuthenticationFilter`	`centerId` claim from validated JWT

MDC Log Correlation (auth-svc)

Within auth-svc, the `JwtAuthenticationFilter` sets MDC context for structured logging:

```
MDC.put("userId", principal.userId().toString());
MDC.put("role", principal.role().name());
// ... request processing ...
MDC.remove("userId");
MDC.remove("role");
```

Combined with the `X-Request-ID` header and OpenTelemetry trace propagation, every log line can be correlated back to a specific user and request.

---

11. Audit Trail

Every significant authentication, enrollment, financial, and assessment event is published to the `audit-immutable` Kafka topic in addition to the service-specific topic.

Properties

- **Immutable append-only:** No update or delete operations ever target the `audit-immutable` topic. Records are written once and retained indefinitely.
- **Best-effort delivery:** Audit event publication failures are logged at ERROR level but do not roll back the primary database transaction. The primary business operation always takes precedence.
- **Event coverage:** All 21 domain events across 6 services are routed to `audit-immutable`. See KAFKA_EVENT_CATALOG.md for the complete list.

Authentication Events Specifically

The following events form the authentication audit chain:

EVENT	WHAT IT RECORDS
`UserRegisteredEvent`	Account creation: userId, email, role, centerId
`UserAuthenticatedEvent`	Every login attempt: success/failure, IP, userAgent
`TokenRefreshedEvent`	Token rotation: old and new tokenId for replay detection
`UserLogoutEvent`	Session termination: single or all sessions
`OtpRequestedEvent`	OTP generation: email, purpose, delivery channel (no OTP value)

Correlating these events by `userId` and `occurredAt` enables:

- Detection of credential stuffing (many `UserAuthenticatedEvent` with `success=false` for the same email)
- Detection of refresh token reuse attacks (same `oldTokenId` appearing in multiple `TokenRefreshedEvent` records)
- Complete session lifecycle reconstruction for forensic analysis

---

12. Threat Model Summary

THREAT	ATTACK VECTOR	MITIGATION
JWT theft	Token intercepted from network or client storage	15-min access token TTL limits window of use; device fingerprint (`deviceFP` claim) makes stolen token unusable from different device
Credential stuffing	Automated login attempts with breached credential lists	Argon2id (computationally expensive hash) + hCaptcha bot detection + Resilience4j application-level rate limiting + gateway token bucket rate limiting
Token replay	Attacker replays a previously valid access or refresh token	Access tokens expire after 15 min; refresh tokens are single-use (deleted from Redis on consumption); `TokenRefreshedEvent` with `oldTokenId` enables reuse detection

THREAT	ATTACK VECTOR	MITIGATION
Refresh token theft	Refresh token extracted from secure storage or transit	Device fingerprint bound: refresh token rejected if `deviceFP` in Redis does not match the requesting device; Redis SSL prevents transit interception
Privilege escalation	Client supplies forged `X-User-Role` header	Gateway overwrites all `X-User-*` headers from JWT claims after validation; clients cannot supply these headers directly
SQL injection	Malicious input in query parameters or request body	JPA/JPQL exclusively; no string-concatenated SQL; parameterized queries enforced by Hibernate
DoS / traffic flooding	High-volume requests from single origin	Gateway-level Redis token bucket (configurable burst + replenish rate) returns HTTP 429; AI endpoint per-requester limit (60 req/min)
AI abuse / cost amplification	Excessive LLM API calls from a single requester	`RedisRateLimitAdapter` enforces 60 req/min per requester+model combination in ai-gateway-svc
Data exfiltration via deletion	Attacker triggers data deletion to cover tracks	Soft delete only — `deleted_at` timestamp set, record retained; physical deletion not possible through application layer
Insider threat / accidental over-privilege	Service-to-service credential reuse	Per-service database credentials; no shared superuser; each service schema is isolated
Session persistence after logout	Logout does not invalidate existing tokens	Refresh token deleted from Redis on logout; access tokens expire after 15 min; logout-all deletes entire `rt:user:{userId}` set
Man-in-the-middle	Network interception of service-to-service calls	HTTPS in production; Istio mTLS planned for full mutual authentication between services
Bot-driven account creation	Automated registration to abuse platform resources	hCaptcha Enterprise on registration endpoint; server-side verification with circuit breaker protection
OTP brute force	Automated guessing of 6-digit OTP	Max attempts counter in Redis (default: 3); OTP deleted after max attempts; OTP expires after configurable TTL (default: 5 min)
Audit log tampering	Modifying or deleting audit records	`audit-immutable` Kafka topic — no application-layer update/delete path; append-only by design